

Paper 1: Rhizoh Engine — An Epistemic Governance Runtime for Large Language Model Execution

Author: Can Kasaplar

Domain: Autonomous Systems Security, LLM Safety, Epistemic Uncertainty, Runtime Verification

Abstract

As Large Language Models (LLMs) transition from conversational agents to execution runtime engines, traditional access controls and dynamic alignment guards fail to address non-deterministic tool invocation, context poisoning, and privilege escalation. Existing safety frameworks evaluate outputs post-generation without formal runtime guarantees on system state mutation.

We present **Rhizoh Engine**, a model-agnostic, deterministic **Epistemic Governance Runtime**

Layer that intercepts candidate execution intents I_t before state mutation occurs. Rhizoh evaluates intents against a dynamic Epistemic Uncertainty metric $U(t) = f(C, A, P, T)$, an Open Policy Agent (OPA) engine, and cryptographic capability tokens. We define governance as a formal **State Transition System** $G(I, S) \Rightarrow$, mapping inputs to execution (δ), human-in-the-loop pause (ρ), or zero-mutation rejection (S_t). We provide formal mathematical proofs for four core invariants (**Zero-Mutation**, **Risk Monotonicity**, **Deterministic Execution**, and **Decision Matrix Soundness & Completeness**) and a cryptographic **Provenance Integrity Lemma**. Furthermore, we establish a computational complexity analysis proving that, conditioned on an extracted candidate intent I_t , governance execution complexity is bounded by $\mathcal{O}(|V_{\text{req}}| + R \cdot D + K)$, independent of underlying LLM parameter scale or prompt window size. Finally, we present a reference implementation (rhizoh-runtime v0.1) featuring cryptographic Merkle tree provenance logs.

1. Introduction & Design Principles

Autonomous LLM agents invoke external tool APIs, mutate databases, and execute code blocks dynamically. Existing safety approaches suffer from two fundamental design flaws:

1. **Probabilistic Post-Inference Guardrails:** Checking safety on generated natural language text offers no deterministic guarantees over execution side-effects.

2. **Static Identity Access Management (IAM):** Static authorization models cannot evaluate runtime context decay, conflicting external inputs, or epistemic uncertainty.

To address these limitations, Rhizoh Engine is built on six core design principles:

- **Zero Trust Boundary:** Raw prompts and model outputs are treated as untrusted intent candidates (I_t).
- **Pre-Mutation Interception:** Governance enforcement occurs strictly prior to state transition dispatch $\delta(S_t, a)$.
- **Tri-Factor Matrix Evaluation:** Decoupled authorization across Policy (OPA), Capability Tokens (Macaroons), and Epistemic Uncertainty ($U(t)$).
- **Deterministic Execution:** Eliminates stochastic decision-making at the governance layer.
- **Cryptographic Non-Repudiation:** Merkle tree provenance structures capture end-to-end execution evidence in an append-only graph.
- **Model Agnosticism:** Complete architectural decoupling from underlying LLM model weights or prompt formats.

2. Related Work & Literature Gap Taxonomy

2.1 Post-Generation Guardrails vs. Pre-Mutation Governance

Existing LLM safety frameworks heavily rely on post-inference text moderation or secondary model validation (e.g., Guardrails AI, NeMo Guardrails). While effective for conversational alignment, these approaches introduce non-deterministic evaluation and scale latencies

proportionally to model inference (T_{LLM}). More critically, they operate on generated text tokens rather than structured execution intents. In contrast, Rhizoh Engine introduces a strict

Pre-Mutation Interception Boundary, isolating candidate intents I_t prior to sandbox dispatch and ensuring runtime latency bounds $\mathcal{O}(|V_{req}| + R \cdot D + K)$ that are invariant to LLM parameter scale.

2.2 Static Access Control vs. Dynamic Epistemic Governance

Declarative policy engines such as Open Policy Agent (OPA) and AWS Cedar enforce deterministic access control based on static role mappings. However, in autonomous agent runtimes, privilege escalation frequently occurs not through missing permissions, but through context poisoning, stale retrieval data, or semantic contradictions. Rhizoh Engine bridges this fundamental gap by combining deterministic Rego policy AST evaluation with a dynamic

Epistemic Uncertainty Engine ($U(t) = f(C, A, P, T)$), ensuring that authorization requires both policy compliance and contextual completeness.

2.3 Theoretical State Semantics in Agent Runtimes

While sandbox frameworks (e.g., ToolEmu, SWE-bench runtimes) simulate agent tool interactions, they lack formal state transition guarantees. Rhizoh Engine formalizes execution governance as a state transition system $G(I_t, S_t) \implies S'$, establishing operational proofs for zero-mutation under rejection ($\text{Reject} \implies S' = S_t$) and risk monotonicity under uncertainty growth.

3. Threat Model & Attack Space Decomposition

We define the total attack surface $\mathcal{A}_{\text{total}}$ as:

$$\mathcal{A}_{\text{total}} = \mathcal{A}_{\text{prompt}} \cup \mathcal{A}_{\text{context}} \cup \mathcal{A}_{\text{capability}} \cup \mathcal{A}_{\text{execution}} \cup \mathcal{A}_{\text{model}} \cup \mathcal{A}_{\text{hardware}}$$

Rhizoh Engine explicitly establishes a formal boundary between mitigated in-scope execution threats ($\mathcal{A}_{\text{in}}$) and out-of-scope substrate risks ($\mathcal{A}_{\text{out}}$):

$$\mathcal{A}_{\text{in}} = \mathcal{A}_{\text{prompt}} \cup \mathcal{A}_{\text{context}} \cup \mathcal{A}_{\text{capability}} \cup \mathcal{A}_{\text{execution}}$$

$$\mathcal{A}_{\text{out}} = \mathcal{A}_{\text{model}} \cup \mathcal{A}_{\text{hardware}}$$

Attack Vector	Source Layer	Mitigation Mechanism	Formal Invariant Bound
Indirect Prompt Injection	External Data / Tool Input	Pre-execution Intent Interception & Policy Check	Theorem 1 (Zero-Mutation)
Context Poisoning / Stale Data	Retrieval Context	Epistemic Uncertainty Threshold ($U(t) > \quad$)	Theorem 2 (Risk Monotonicity)
Privilege	Tool Execution	Macaroon	Theorem 4

Escalation	Scope	Capability Caveat Verification	(Decision Matrix Soundness)
Audit Log Tampering	Provenance System	SHA-256 Binary Merkle Tree Construction	Lemma 1 (Provenance Integrity)

4. Formal System Semantics & Epistemic Formulation

4.1 State Transition Governance System

Let $S_t \in \mathcal{S}$ be the current system state, and $I_t \in \mathcal{I}$ be the candidate execution intent. The required evidence sub-graph V_{required} is derived via the evidence operator Γ :

$$V_{\text{required}} = \Gamma(I_t, S_t, \mathcal{P})$$

The governance mapping $G : \mathcal{I} \times \mathcal{S} \times \mathcal{P} \rightarrow \mathcal{S}'$ is formally defined as:

$$G(I_t, S_t, \mathcal{P}) = \begin{cases} \delta(S_t, a), & \text{if Policy}(I_t, \mathcal{P}) \wedge \text{Cap}(K_{\text{cap}}, I_t) \wedge (U(t) \leq \theta_{\text{risk}}) \implies \text{Allow} \\ \rho(S_t, I_t), & \text{if } U(t) > \theta_{\text{risk}} \wedge \text{Policy}(I_t, \mathcal{P}) \implies \text{Pause (Human-in-the-loop)} \\ S_t, & \text{otherwise} \implies \text{Reject (Zero Mutation)} \end{cases}$$

4.2 Epistemic Uncertainty Formulation & Weight Calibration ($U(t)$)

Epistemic uncertainty $U(t) \in [0, 1]$ models system ignorance regarding context completeness, staleness, contradictions, and source trust:

$$U(t) = f(C, A, P, T) = w_1(1 - C) + w_2A + w_3P + w_4(1 - T)$$

where $\sum_{i=1}^4 w_i = 1.0$, $w_i \geq 0$, and individual parameters represent:

- **Coverage** ($C \in [0, 1]$): Ratio of present evidence nodes $|V_{\text{present}}|$ to required nodes

$|V_{\text{required}}|$.

- **Staleness** ($A \in [0, 1]$): Timestamp age decay of context evidence.
- **Conflict** ($P \in [0, 1]$): Semantic contradiction index across active evidence nodes.
- **Trust** ($T \in [0, 1]$): Cryptographic signature confidence score of evidence sources.

Weight Calibration Note: In this reference specification, initial weights

$w = (0.4, 0.2, 0.2, 0.2)$ are assigned as domain-expert calibrated defaults

prioritizing coverage completeness ($w_1 = 0.4$) over temporal decay. Sensitivity analysis under varied weight vectors is evaluated empirically in Paper 2, while dynamic Bayesian weight optimization is formally deferred to future work.

5. Formal System Invariants & Mathematical Proofs

Theorem 1 (Zero-Mutation Operational Invariant)

Statement: Under the operational semantics defined in Section 4, for any candidate intent I_t and initial state S_t , if $G(I_t, S_t, \mathcal{P}) = \text{Reject}$, the post-governance state S' strictly satisfies $S' =$.

Proof:

Let $a \in \text{Sandbox}$ be dispatched if and only if the decision matrix evaluates to Allow .

When $G(I_t, S_t, \mathcal{P}) = \text{Reject}$, the execution dispatcher returns $a = \emptyset$. Since no state transition container is instantiated, the system state transition function defaults to:

$$S' = \delta(S_t, \emptyset) = S_t$$

Under an isolated sandbox assumption, zero state mutation holds strictly for all rejected requests. ■

Theorem 2 (Risk Monotonicity Invariant)

Statement: Let U_1, U_2 be epistemic uncertainty values evaluated for a candidate intent I_t . If

$U_1 \geq U_2$, then $\text{Risk}(U_1) \geq \text{Risk}(U_2)$.

Proof:

The risk evaluation function $R(U) = \mathbb{I}(U > \theta_{\text{risk}})$ is a monotonically non-decreasing indicator step function. Suppose $U_1 \geq U_2$.

1. If $U_2 > \theta_{\text{risk}}$, then $U_1 \geq U_2 > \theta_{\text{risk}}$, yielding $R(U_1) = R(U_2) = 1$.
2. If $U_1 \leq \theta_{\text{risk}}$, then $U_2 \leq U_1 \leq \theta_{\text{risk}}$, yielding $R(U_1) = R(U_2) = 0$.
3. If $U_2 \leq \theta_{\text{risk}} < U_1$, then $R(U_1) = 1 > R(U_2) = 0$.

In all cases, $U_1 \geq U_2 \implies R(U_1) \geq R(U_2)$. ■

Theorem 3 (Deterministic Execution Invariant)

Statement: Given a fixed input tuple $(I_t, S_t, \mathcal{P})$ and assuming deterministic policy inputs, the governance decision $G(I_t, S_t, \mathcal{P})$ is strictly deterministic.

Proof:

Governance evaluation relies on three sub-functions:

1. $U(t) = w_1(1 - C) + w_2A + w_3P + w_4(1 - T)$, consisting of deterministic linear algebra and timestamp arithmetic.
2. OPA Rego AST evaluation $O(I_t, \mathcal{P})$, which is deterministic and side-effect-free by language specification under deterministic policy inputs.
3. Cryptographic Ed25519 signature and HMAC capability verification $V(K_{\text{cap}})$, which is deterministic.

Since all constituent operators are deterministic under side-effect-free state copies,

$G(I_t, S_t, \mathcal{P})$ produces identical decision outcomes under identical input tuples. ■

Theorem 4 (Governance Decision Matrix Soundness & Completeness)

Statement: For any candidate intent I_t , initial state S_t , and policy set $\mathcal{P}$:

$$G(I_t, S_t, \mathcal{P}) = \text{Allow} \iff \text{PolicyCheck}(I_t, \mathcal{P}) \wedge \text{CapabilityCheck}(K_{\text{cap}}, I_t) \wedge (U$$

Proof:

- ($\implies$ Decision Matrix Soundness): System dispatch is guarded by a conjunction operator $\bigwedge_{i=1}^3 P_i$. If any predicate $P_i \in \{\text{PolicyCheck}, \text{CapabilityCheck}, \mathbb{I}(U(t) \leq \theta_{\text{risk}})\}$ evaluates to **False**, the governance state machine transitions to either **Reject** or **Pause**. Therefore, $G(I_t, S_t, \mathcal{P}) = \text{Allow}$ implies all three predicates evaluate to **True**.
- ($\longleftarrow$ Decision Matrix Completeness): If all three predicates evaluate to **True**, the decision matrix defaults to state transition $\delta(S_t, a)$, which yields $G(I_t, S_t, \mathcal{P}) = \text{Allow}$. Within the governance decision matrix boundary, the condition is necessary and sufficient. ■

Lemma 1 (Cryptographic Provenance Integrity & Tamper-Evident Log)

Statement: Let $\mathcal{M}_n = H(H_{n-1} \parallel e_n)$ be the Merkle DAG node root hash representing an ordered sequence of governance audit events $E = (e_1, e_2, \dots, e_n)$, where H is a collision-resistant cryptographic hash function (SHA-256). Any retroactive alteration, insertion, or deletion of an event e_i ($1 \leq i \leq n$) modifies the root hash $\mathcal{M}_n$ with probability $1 - 2^{-256}$.

Proof:

By definition of cryptographic hash collision resistance, finding $e'_i \neq e_i$ such that $H(e'_i) = H(e_i)$ requires $\mathcal{O}(2^{128})$ operations under the birthday attack bound. Because each ancestor node $H_k = H(H_{k-1} \parallel e_k)$ recursively binds e_i for all $k \geq i$, changing any historical entry e_i propagates modifications up the parent trajectory to $\mathcal{M}_n$. Thus, the provenance graph forms an append-only, tamper-evident log structure. ■

6. Computational Complexity Analysis

Conditioned on an extracted candidate intent I_t , total governance execution time $T_{\text{Governance Runtime}}$ is bounded by:

$$\mathcal{O}(|V_{\text{required}}| + R \cdot D + K)$$

where:

- $|V_{\text{required}}|$: Number of required evidence graph nodes evaluated during Γ operator evaluation.
- $R \cdot D$: Number of policy rules (R) multiplied by maximum Rego AST rule evaluation depth (D) (referencing standard OPA AST evaluation bounds).
- K : Number of cryptographic caveat checks evaluated during capability verification.

Asymptotic Scale Invariant: Post-intent extraction, this bound is $\mathcal{O}(1)$ with respect to LLM parameter scale (N_{params}) and input prompt token length (L_{tokens}), strictly isolating governance runtime overhead from model generation latency.

7. Discussion — The Paradigm Shift in Execution Safety

7.1 Decoupling Safety Verification from LLM Inference

The core paradigm shift introduced by Rhizoh Engine is the structural decoupling of governance verification from LLM model execution. By treating the LLM strictly as an untrusted, non-deterministic intent generator and the governance runtime as a deterministic state mutation gatekeeper, we eliminate the need for costly secondary LLM evaluation calls.

7.2 The Complementary Role to Standard Policy Engines

Rather than replacing established authorization standards, Rhizoh Engine acts as an epistemic layer above engines like OPA. While OPA answers the question *"Is this identity authorized to call this API?"*, Rhizoh answers *"Is the underlying context complete, fresh, and uncontradicted enough to allow state mutation safely right now?"* This complementary positioning allows industrial deployments to leverage existing Rego policy investments while gaining protection against context poisoning attacks.

7.3 Systematic Literature Comparison

Framework / System	Deterministic Policy Engine	Cryptographic Provenance DAG	Epistemic Uncertainty Model $U(t)$	Formal State Transition $G(I, S) \rightarrow S'$	Formal Zero-Mutation Semantics	Overhead Scale Factor
Guardrails AI	$\times$ (Probabilistic)	$\times$	$\times$	$\times$	$\times$	$\mathcal{O}(T_{LLM})$
NeMo Guardrails	$\checkmark$ (Rail Rules)	$\times$	$\times$	$\times$	$\times$	$\mathcal{O}(\text{Rules})$
AWS Cedar / OPA	$\checkmark$ (Static Auth)	$\times$	$\times$	$\times$	$\checkmark$	$\mathcal{O}(\text{AST})$
ToolEmu	$\times$	$\times$	$\times$	$\checkmark$ (Simulation)	$\times$	$\mathcal{O}(T_{LLM})$
Rhizoh Engine	$\checkmark$ (OPA / Rego)	$\checkmark$ (Merkle Trees)	$\checkmark$ ($f(C, A, F)$)	$\checkmark$ ($G(I, S)$)	$\checkmark$ ($S' =$)	$\mathcal{O}(V +$

8. Conclusion

Paper 1 establishes the formal semantics, threat model, and operational proofs for Rhizoh Engine. By proving four core decision invariants, a provenance integrity lemma, and a computational bound independent of LLM parameter scale post-intent extraction, we show that autonomous agent execution governance can be framed as a deterministic, pre-mutation state transition system.